Digital Twin V2

Weekly updates

Week 3-4

Overview

1. Misc bug fixes + improvements
2. Improve temporal consistency of DA3
3. Collect IRL data
4. IRL data processing + more bug fixes

Bug fixes and Improvement

Previous pipeline had *hardcoded* image sizes and video sizes for NO reason

- Several components (cotracker, shape prior generation) prefer higher res.

Problem with new pipeline:

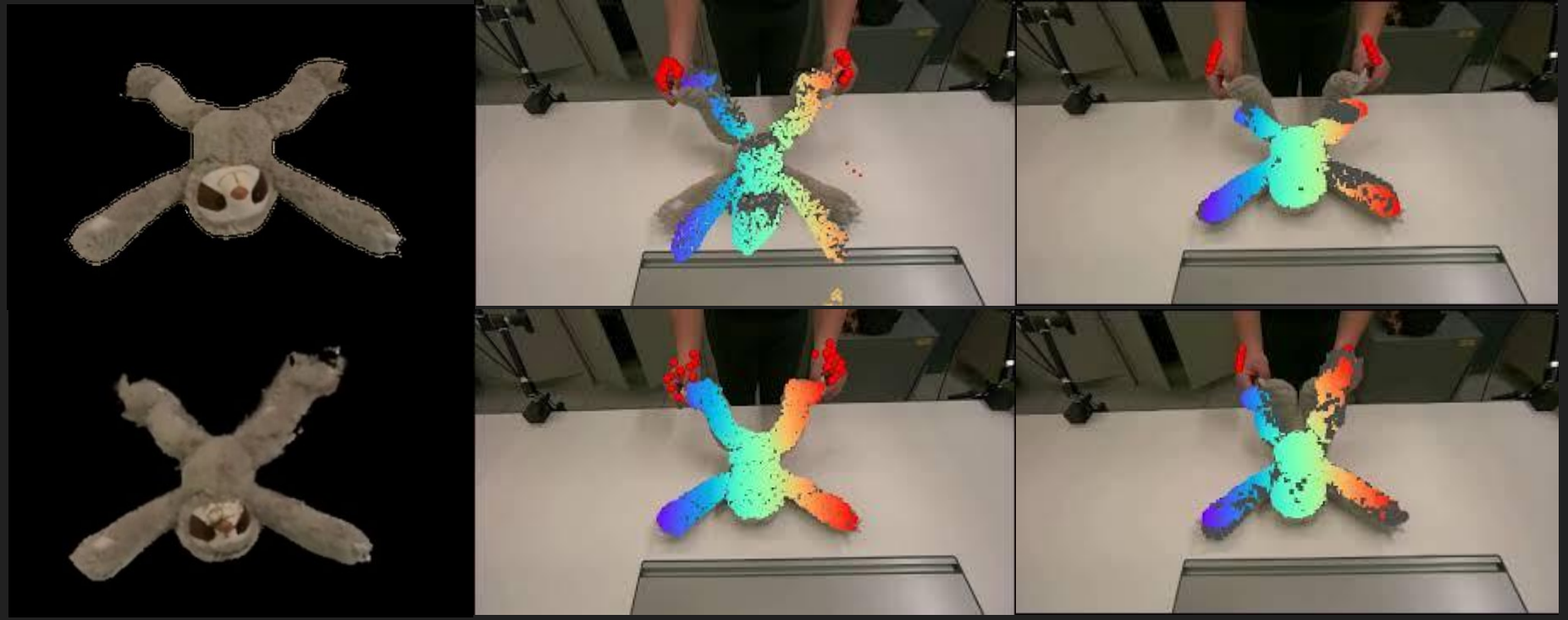
- Collected video is 1080p
- DA3 depth maps are 280 x 504

The fix: allow flexible video, depth, and image sizes

Also fixed some mistakes with camera intrinsics/extrinsics handling…

And some things with controller point attachment…

General improvements



DA3 Temporal Consistency

DA3 inference per-frame, and reproject to form 3 depth maps

- Jitter between frames
- Inconsistent depth boundaries between frames

Resulted in:

- Poor object reconstruction
- Poor training data with lots of noise

DA3 with Streaming

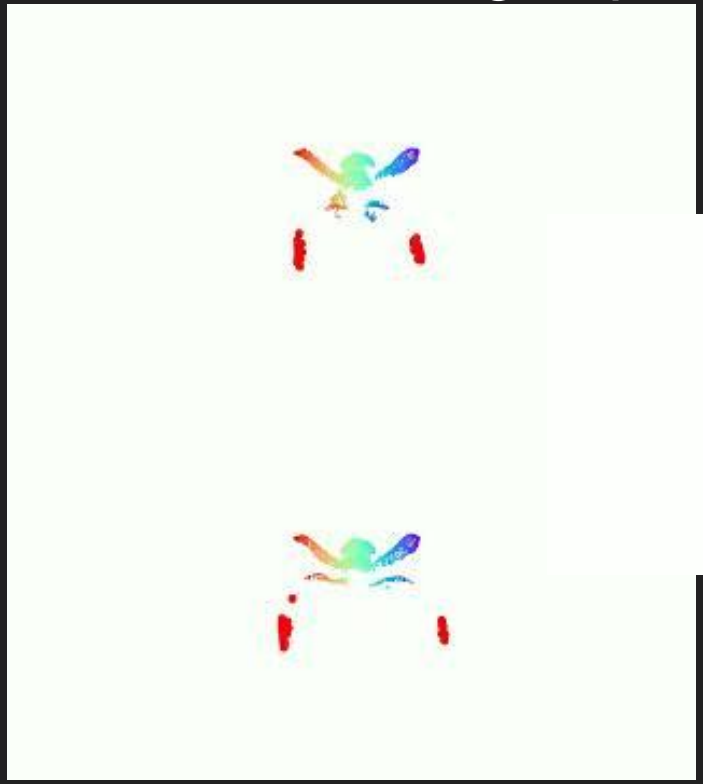
Solution: enable streaming for DA3 (adapted code from da3-streaming to work with multiple cameras)

- Process chunks of frames with overlap
- Share weights from overlapped frames to improve temporal consistency

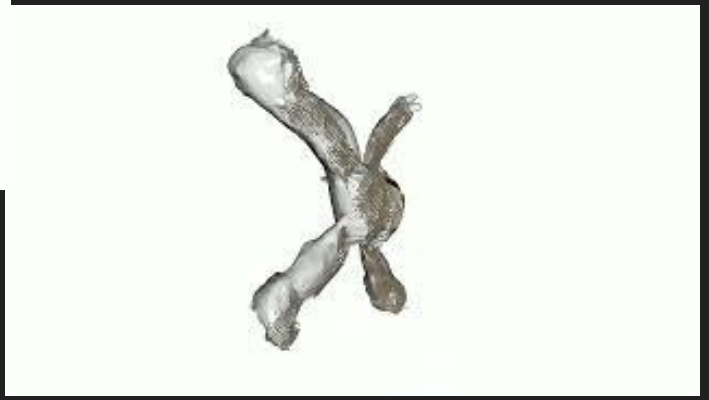
Remaining Issues:

- Larger chunk sizes cause alignment failure in da3 (something about singular matrices in umeyama alignment…)

DA3 streaming improvements

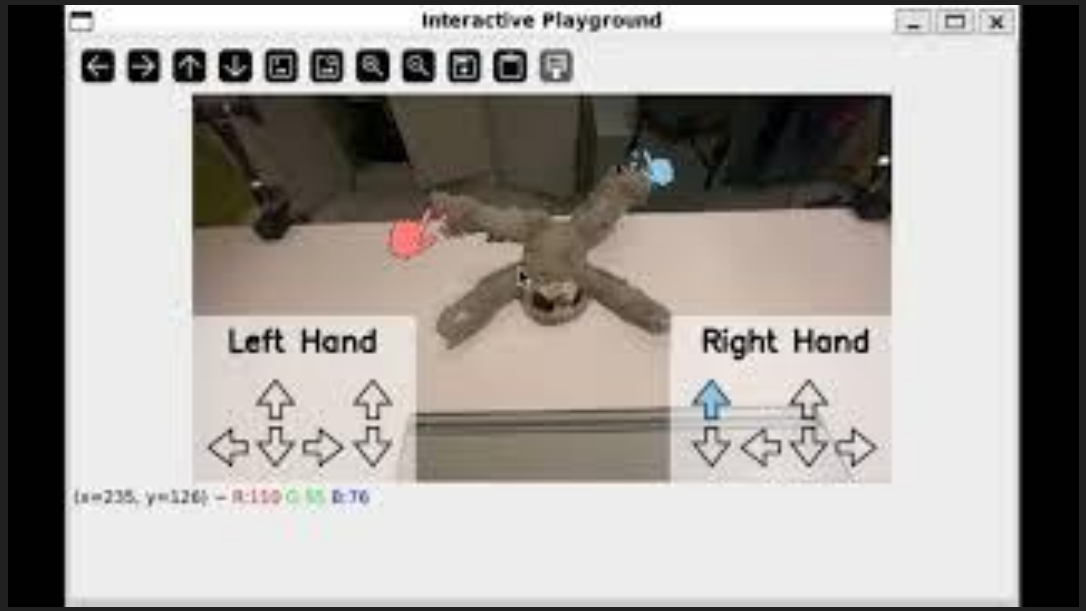






Sloth Model with Modified Pipeline

The WARP model in their interactive playground:



IRL data collection

Collected ~8 different interactions IRL using normal cameras

- Tried a variety of objects:
  - Soft objects: mat, stuffed animals, sloth
  - Hard objects: shoe, various bottle, etc

- **Did not collect camera extrinsics or intrinsics**
  - DA3 pipeline should be able to infer this data from first frames
  - Also all of the cameras have different intrinsics (different cameras)

IRL data!







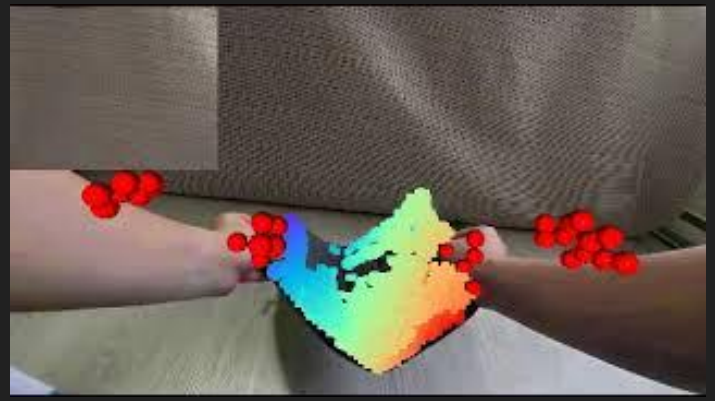
Observations so Far (some fixed)

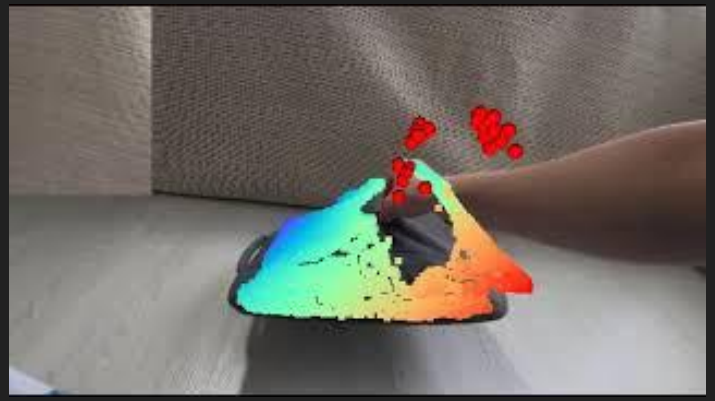
1. Masking is better with more descriptive names, still need manual intervention
2. Shape prior alignment (ARAP) fails for non watertight meshes
3. Pipeline assumes specific position ranges
4. Controller point motion filtering is too strict
5. Dense motion tracking (Cotracker) fails when object is cut off
6. CMA optimization stage diverges (?)
7. Quality of depth data could be improved further

Conclusion: pipeline is brittle and breaks randomly :(

Tracking Results

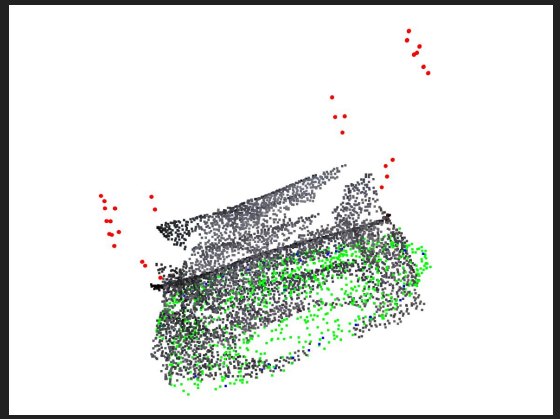
Doesn’t look too bad… has some strange artifact though

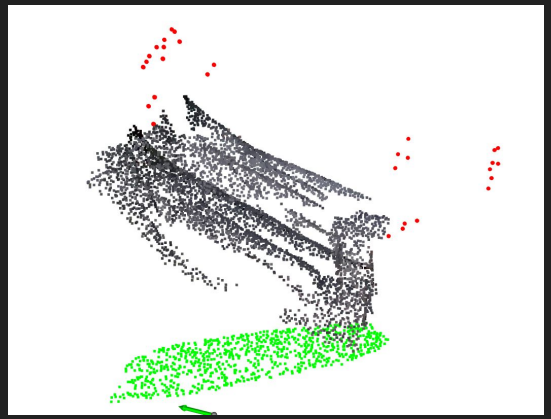




ARAP alignment fix

Added a default fallback that just applied average translation on the tracked points.





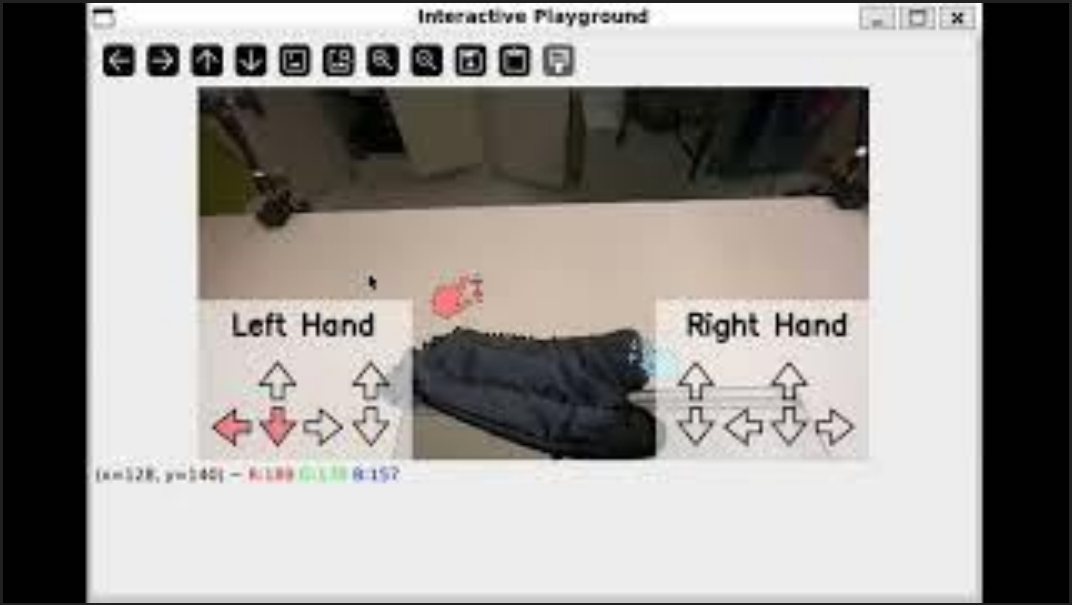
Segmented object due to interaction

This is why cotracker fails:



The Broken WARP model so far…

Something goes wrong during model training



What’s next…

Get a working WARP model from newly collected data!

Week 5-6

Overview

1. Fixed WARP model training issues
2. Improved pose estimation (camera intrinsics/extrinsics)
3. Fixed various bugs in pipeline and improve flow
    - CoTracker resolution issue
    - Incorrect handling for higher resolution RGB images
4. General code refactoring

We now have a good final model directly from data!

From Last Time (1)

Dense motion tracking (Cotracker) fails when object is cut off

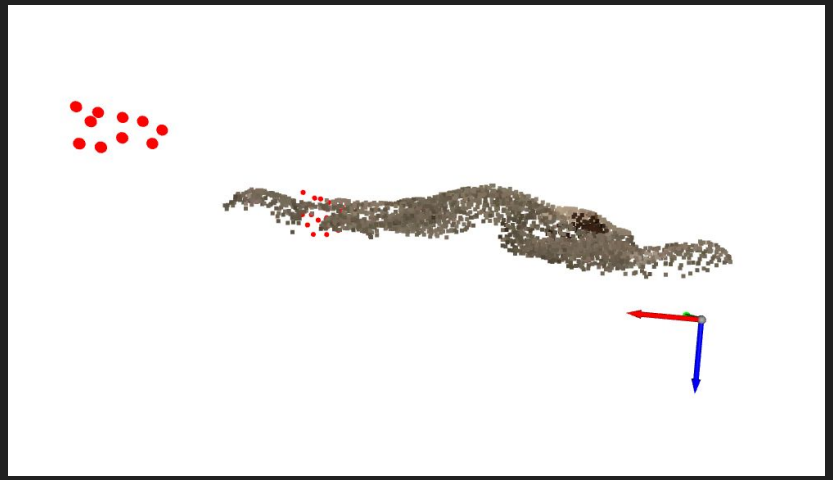
- Used morphological occlusion to join disjoint sections of masks



From Last Time (2)

CMA optimization stage diverges (?)

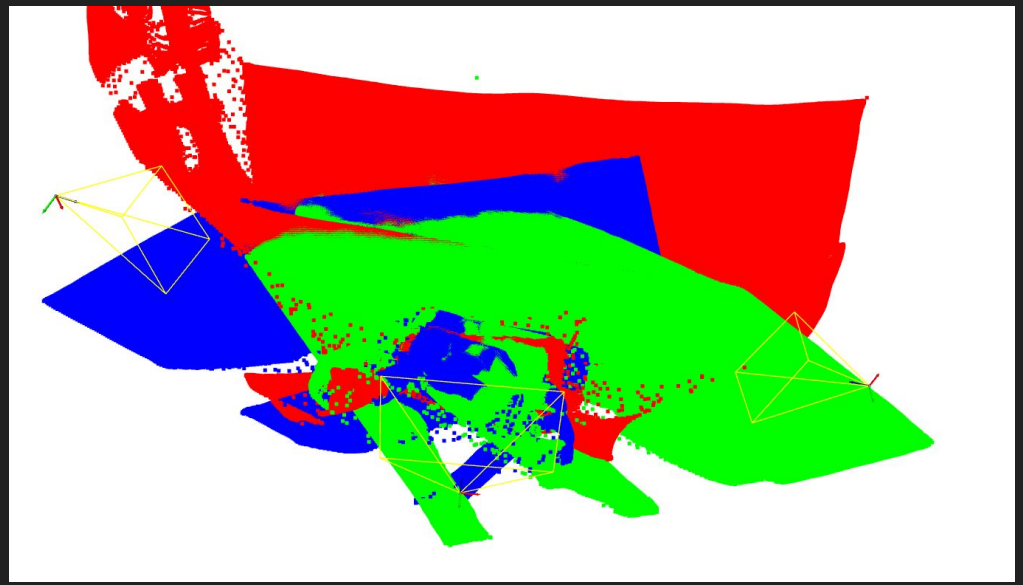
- Created new extrinsics alignment stage to ensure object is above ground
- Propagate alignment to data tracks and meshes



From Last Time (3)

Quality of depth data could be improved further

- Added a better pose estimation step with DUSt3R



Pose Estimation

DA3 pose estimation is pretty bad (both intrinsics and extrinsics were off!)

- Misaligned 3d projections and frames
- Poor optimization and training performance

Separate pose estimation step with DUSt3R + heuristic scale

- Builds 3D correspondence between image pairs
- Has much better 3D alignment compared to DA3

Bugs bugs bugs…

My code was NOT idempotent! (results would be different depending on how many times the pipeline was run)

- Incorrect handling of higher resolution RGB inputs
- Improper handling of intrinsics and extrinsics (due to resolution)

Resulting in

- Misaligned objects
- Poor gaussian data
- Wrong model!

CoTracker Resolution Bug

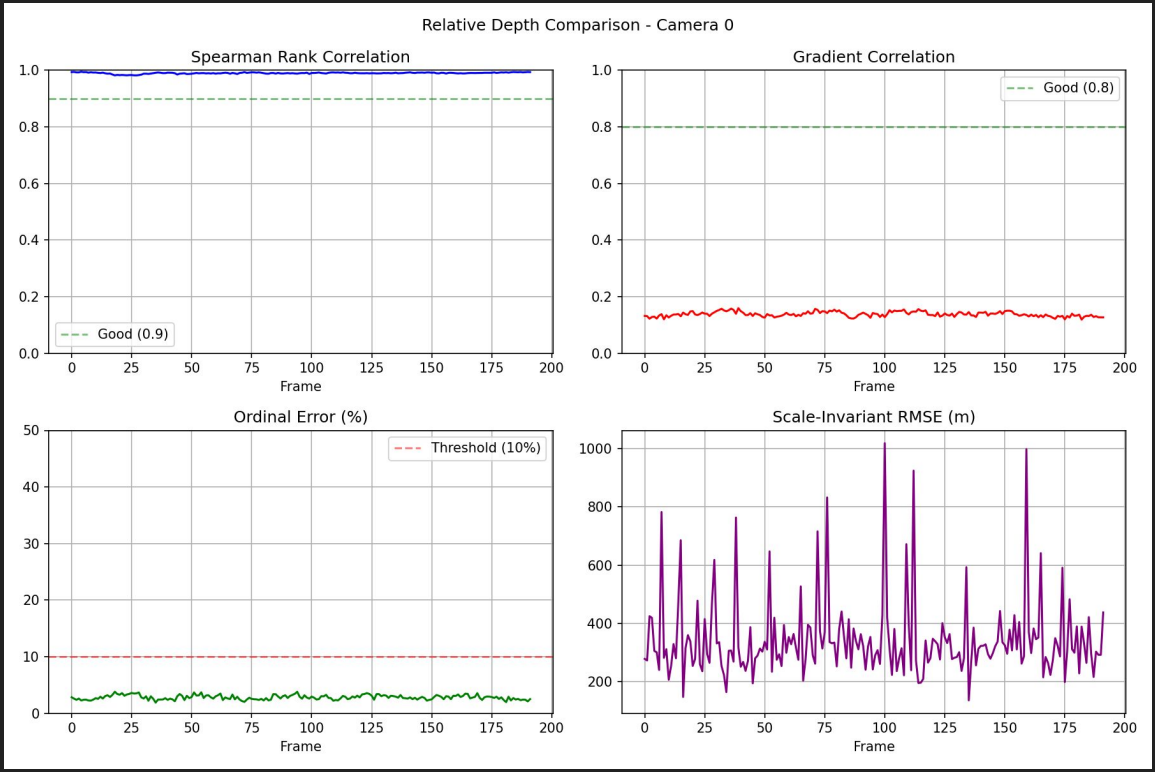
There was a bug where after the first run, camera resolutions would be lowered

- So i never fixed the resolution issue

I did now…

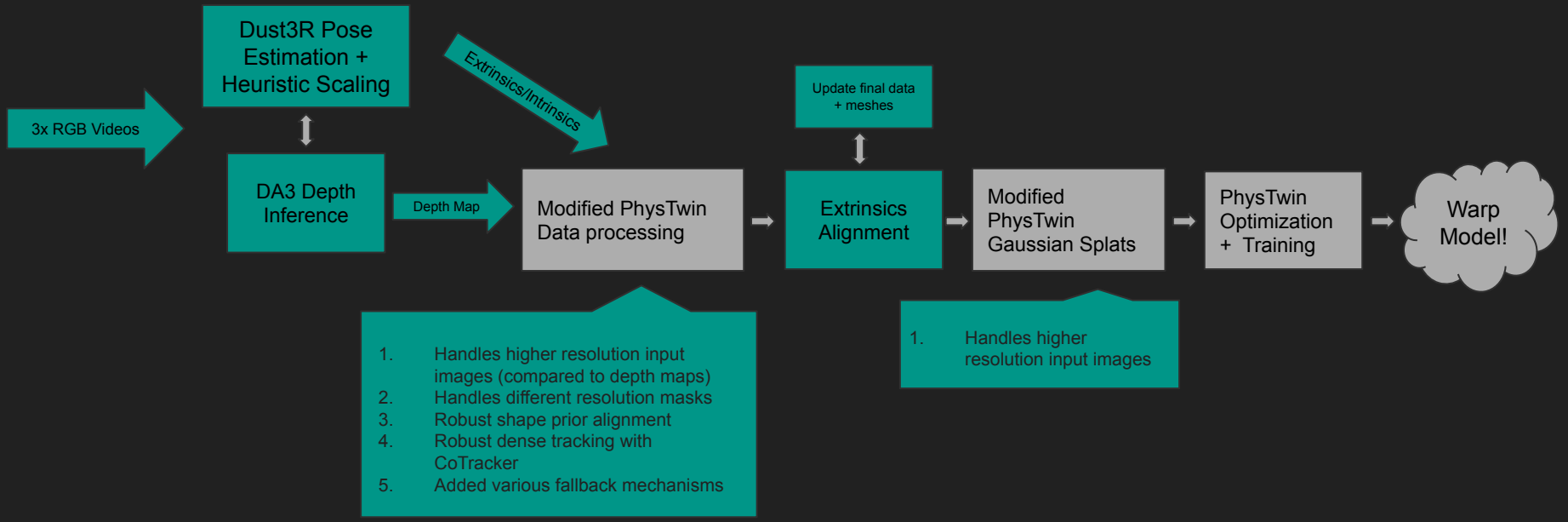
- Higher resolution gives denser tracked points
- Better accuracy
- Less jumpy tracks

Comparing Depth Quality

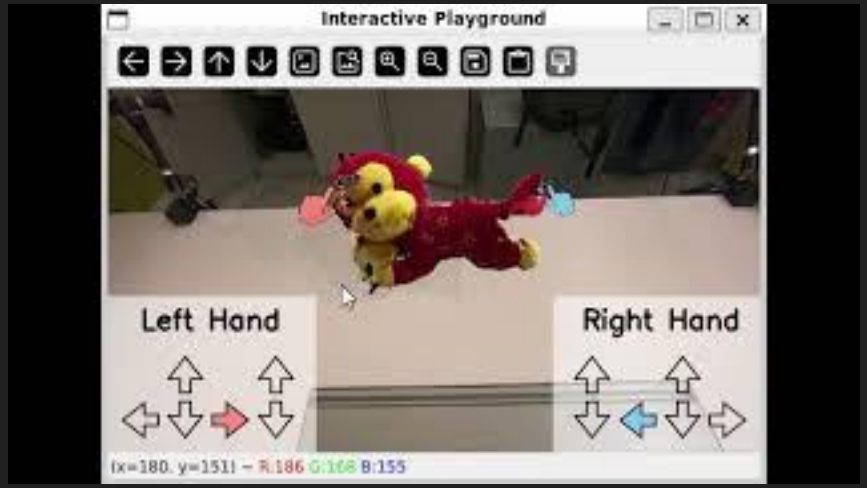



SOURCE RGB
Frame 103
REFERENCE (RealSense)
PREDICTED (Aligned)
RELATIVE ERROR (%)
METRICS
QUALITY METRICS

New Pipeline structure



Results


Interactive Playground
Left Hand
Right Hand
(x=180, y=151) ~ R:186 G:168 B:155


Interactive Playground
Left Hand
Right Hand
(x=219, y=67) ~ R:71 G:67 B:54

More Results



Next Steps and Remaining Issues

We don’t have real streaming

- We have a really bad work around that breaks for larger chunk sizes
- Tried having a better “hack” (Spent a lot of time and it ended up worse)
- Ideally adapt the actual da3_streaming model to work with multi-poses

Look into newer CoTracker, Trellis, and SAM versions for potential improvements

Improve data collection

Improve Prior Alignment

Look into ways to improve depth map 3D coherence

Quick Updates

Trellis and CoTracker are already up to date, no visible improvements

Tried:

- Pi3 model for depth estimation (didn’t work for pose alignment)
- Looked into MVTrack (https://arxiv.org/abs/2508.21060) As a CoTracker replacement, looks promising so far
- Other misc fixes for my code and pipeline
- Tested pipeline on more objects

Findings

- Big, slow moving objects perform well
- Minimal controller point interaction with the object perform well
  - If hand covers a significant part of object it will effect all later stages
  - Effects shape prior generation, alignment, cotracker
- Small, fast moving objects perform poorly!
  - 3d consistency dies when object moves fast
  - Depth map seems inconsistent when object is small (variation is too high)
- Controller must be in contact with object the whole time
  - Or else training results is really bad (this is understandable)

More Findings

My data collection sucks

1. Cameras are different (different focal lengths, distortion, intrinsics, etc)
2. Cameras are not time synced (apparently frame synced != time sync)

This could explain poor 3d reconstruction + inconsistencies at high movements

- I dont have three identical cameras :(
- Pipeline is not robust towards errors like these

Core Problems

1. DA3 has poor spatial consistency on fast moving and small objects
2. DA3 is not temporally smooth

------ results in poor point tracking results with cotracker ----------

Currently

1. Models with good spatial consistency are NOT smooth temporally (Dust3r, MASt3r, DA3 (?))
2. Models with smooth temporal consistency are NOT spatially consistent (or don’t support multi-camera), (VDA, oVDA, etc)

More problems

Pipeline has the following requirements

1. We need to align depth maps to specific camera poses
    a. Grounded SAM needs RGB frames to match depth map frames
    b. Poses + CoTracker to triangulate 3d tracks
2. A lot of 3d construction models DONT align to specific poses
    a. Is smooth and consistent
    b. The og NeRF methods
    c. 4d gaussians
3. A lot of these multi camera models are stationary scene, moving camera
    a. Focus more on single frame scene reconstruction
    b. I guess our use case is more niche?

Potential Solutions?

1. Somehow adapt streaming model to work with multiview + MVTrack
    a. MASt3R/DUSt3R pose inference at beginning
    b. Somehow adapt DA3-streaming to work with multi-views (i have no idea how)
    c. Use a more robust 3D point tracking system (MVTrack or something else a lot of newer options)
    d. May require training a whole new model (out of scope)
    e. This is change is more “aligned” with the phystwin pipeline (which may not be the best)

Some methods to look at: G-CUT3R, R3D3 (old)

Potential Solutions

2. Big pipeline rewrite to use 4DGS, 3D grounded SAM, MVTrack

    a. 4DGS is smooth + 3d consistent
    b. No explicit camera poses anymore (all segmentation, tracking, etc, MUST happen directly in 3d space)
    c. Hence, 3D grounded SAM instead of 2d SAM
    d. Hence, MVTrack instead of CoTracker
    e. This is also hard and might break a lot of things down the line (also a massive pipeline change)

Potential Solutions

Both are pretty hard :(

There is still a lot more research and testing left to do

Hard to balance tradeoffs

IDK if there is an existing model that does EVERYTHING we want

Week 7-8 Updates

Overview

- Testing, testing, testing
  - Tested Pi^3 for depth inference, did not work for our use case
  - Tested MVTrack for 3d tracking, performed much worse
- Comparisons and analysis
  - Comparisons with PhysTwin collected data
  - Comparisons with various types of objects
- Some new problems

So where are we at now?

A functional pipeline that can (kind of) generate working warp models

- Automatic pose inference with DA3 or DUSt3R
- Automatic depth inference with DA3
- Warp model training

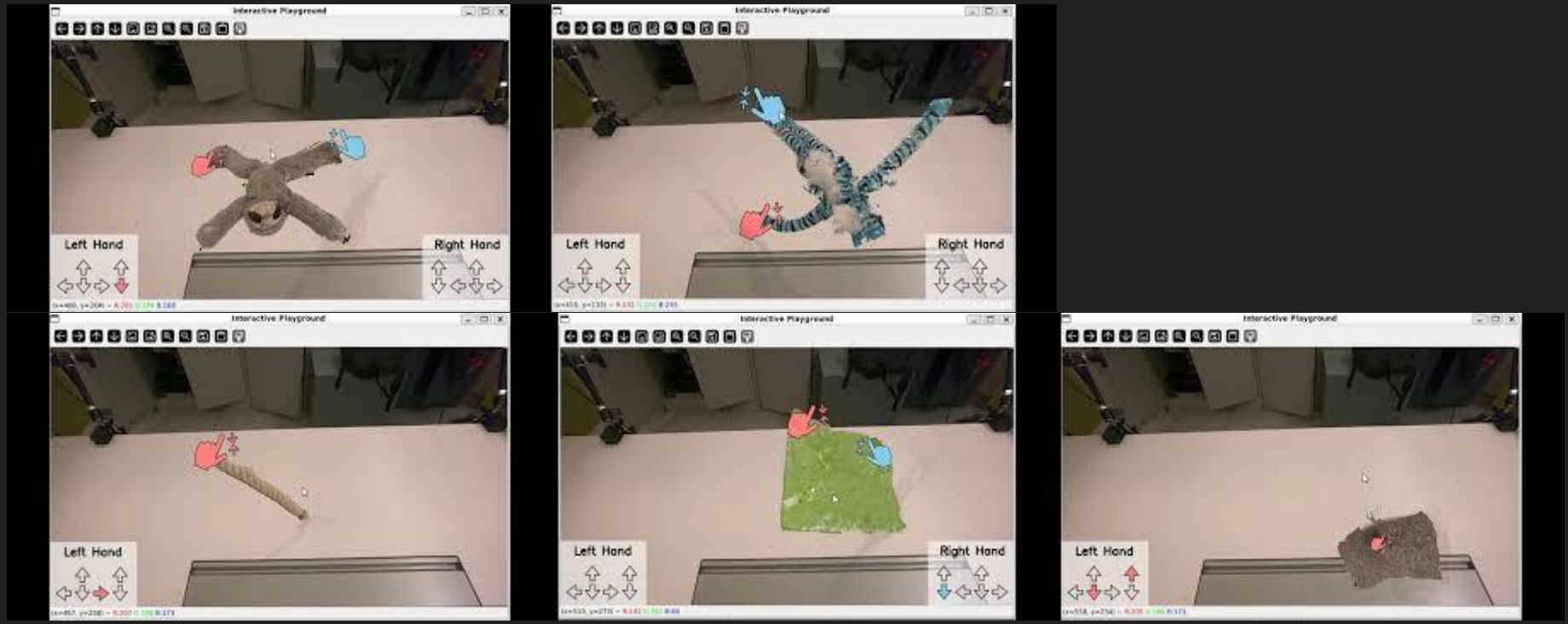
Issues

- Temporal smoothness of DA3
- Occasional spatial inconsistencies
- Data collection problems?
- Does not interact with environment properly

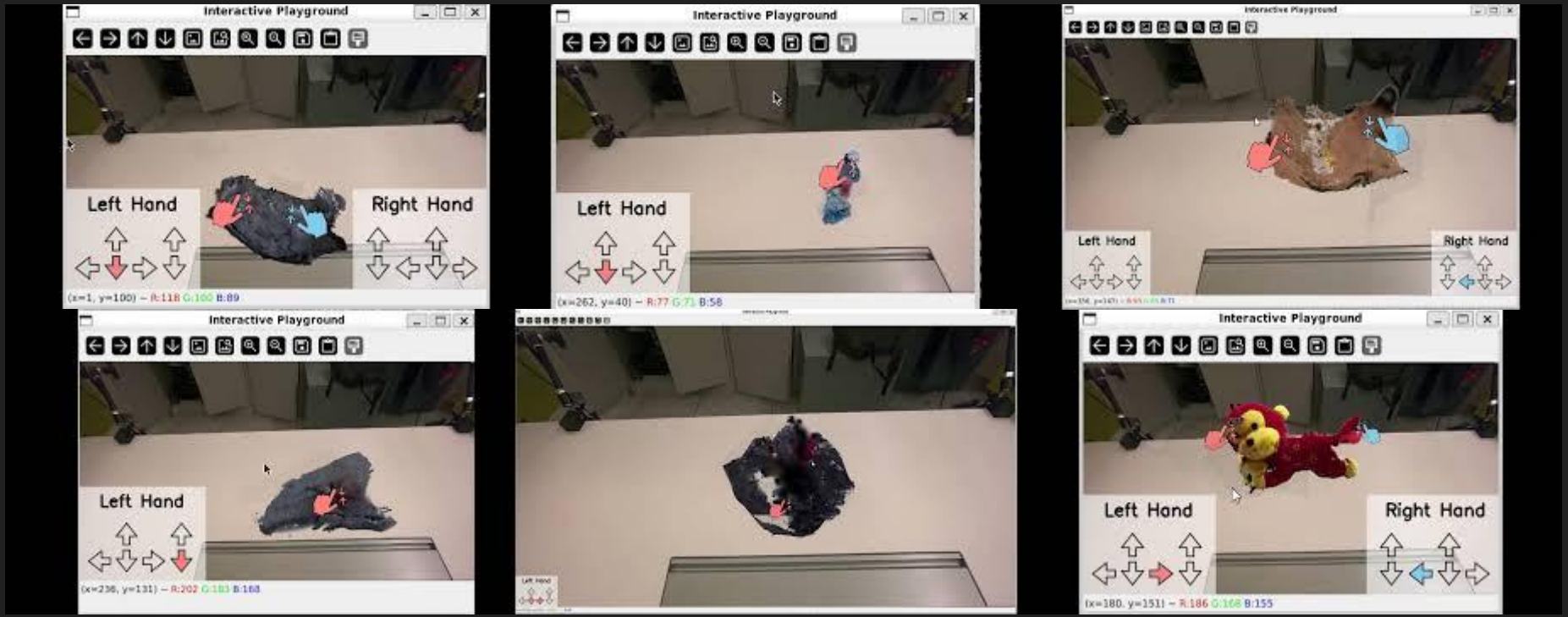
Before all the problems… lets see some cool videos!



Using Phystwin Data



Using Our Data



Findings

- Big, slow moving objects perform well
- Minimal controller point interaction with the object perform well
  - If hand covers a significant part of object it will effect all later stages
  - Effects shape prior generation, alignment, cotracker
- Small, fast moving objects perform poorly!
  - 3d consistency dies when object moves fast
  - Depth map seems inconsistent when object is small (variation is too high)
- Controller must be in contact with object the whole time
  - Or else training results is really bad (this is understandable)

Data Collection Problems?

My setup: 2 phones, 1 DJI nano

- Different intrinsics, different video start times,
- Rolling shutter on all three cameras
- Auto-exposure/focus

This causes

- Flickering?
- Poor spatial consistency as video progresses, or when objects move fast

Spatial Consistency Issues?

We observe spatial separation on fast movements

Spatial coherency decreases overtime

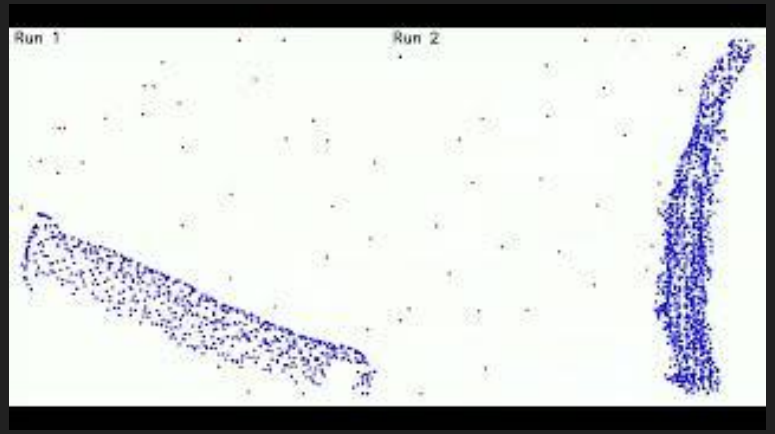
Performs poorly on strange rotations

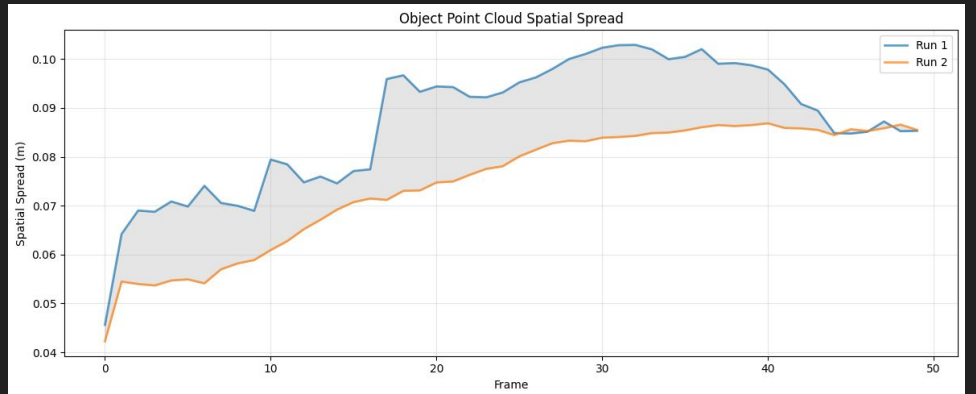
- Could be due to CoTracker

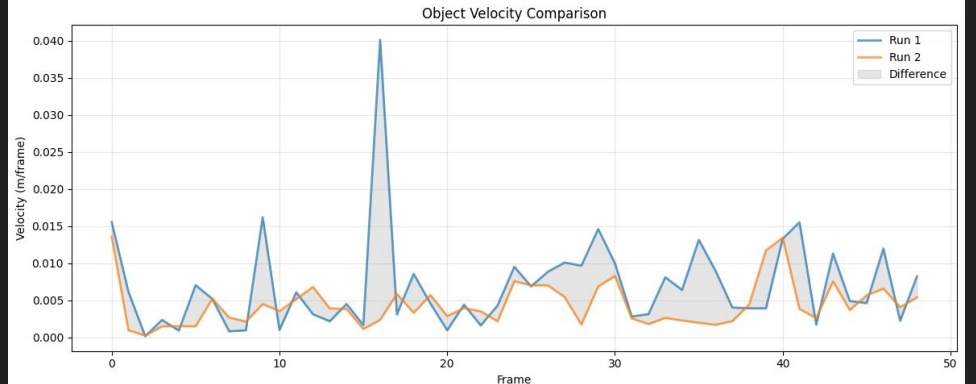


Temporal Smoothness Issues?

Depths flicker. Errors propagate to 3D tracks since we lift from CoTracker 2D.

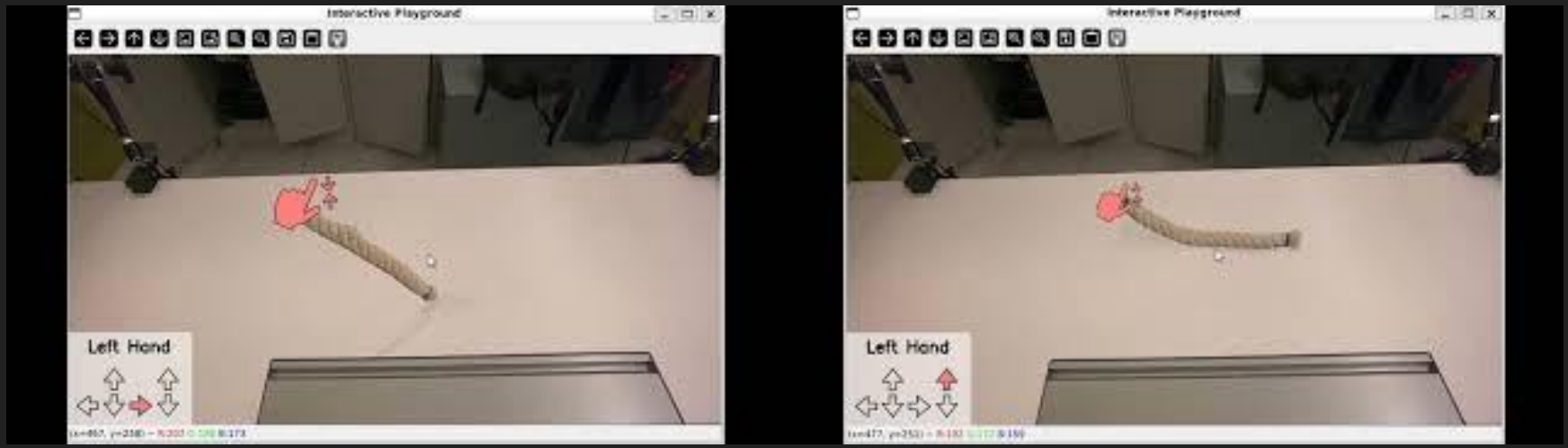






Environment Interaction Issues?

Rope needs to be much more stiff since it has no ground support



Core Limitations

PhysTwin pipelines relies on 2D (segmentation, cotracker)

- Uses depth to lift into 3D, if depth quality is bad everything else fails
- We need a model that gives us good depth quality!

However…

1. Models with good spatial consistency are NOT smooth temporally (Dust3r, MASt3r, DA3 (?))
2. Models with smooth temporal consistency are NOT spatially consistent (or don’t support multi-camera), (VDA, oVDA, etc)

Final (hopefully) Key Tasks

1. **Temporally smooth depth data**
2. **Spatially consistent depth data and poses**
3. **Robust 3D point tracking**
4. **Robust environment interaction**

Solution: Temporally Smooth Depth Data

Post Processing on DA3 data

- DA3 gives us jittery frames
- VDA gives us smooth video (but monocular)

Add some post processing stage (algorithmic or ML based or both)

- Learn some mixture function between VDA depth maps and DA3 depth maps
  - DA3 will provide the baseline
  - VDA will learn the temporal consistency
  - Somehow learn a way to “blend” this
  - (Thanks to Sayem for the suggestion)

Solution: Spatially Consistent Depth and Poses

Collect better data :)

- Use proper time alignment cues (clap, flash, etc)
- Try to use more consistent camera modes (no autofocus, no autoexposure)
- Look into measuring actual extrinsics / intrinsics
    - Intrinsics may be non-trivial and my camera might be slightly fish-eye warped

Solution: Robust 3D point tracking

There are a lot more models to try:

1. Already tried MVTracker (https://ethz-vlg.github.io/mvtracker/), poor results
2. Try SpatialTrackeV2 (https://github.com/henry123-boy/SpaTrackerV2)
3. Do more research here

Solution: Robust Environment Interaction

For now, assuming static environment

- Estimate any floor and wall planes
- Integrate plane colliders in Warp (during training and inference)
- Already working on this

For the future:

- Come up with a more robust solution (treat scene as arbitrary rigid body), maybe the environment changes
- Out of scope

Next steps

Here is what i will focus on in the next 2 weeks:

1. Fix interaction with environment
2. Try to collect better data
3. Investigate potential improvements for temporal smoothness
4. Investigate potential improvements for 3d tracking

Week 9-10 Update

1. Tested SpatialTrackerV2
2. Solved proper ground alignment and ground collisions
3. “Solved” temporal consistency issue
    a. Trained ML model with VDA and DA3 data
4. New data collection!
    a. Set up depth cameras
    b. Collected ~30 different sessions

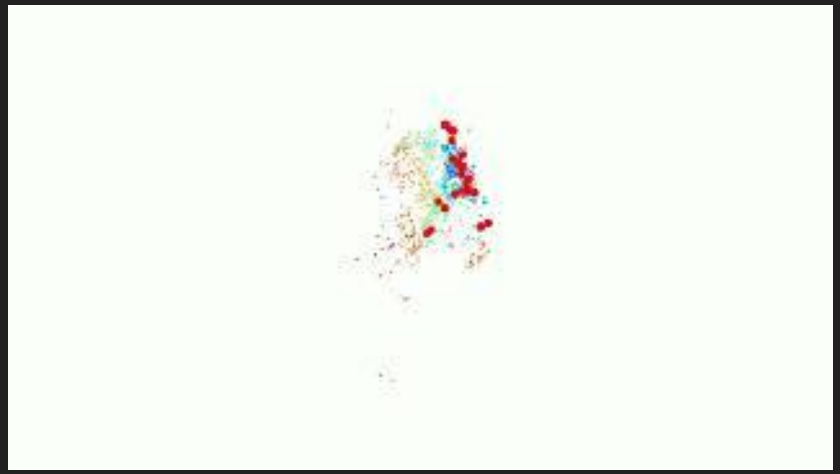
From Last Time

1. Temporally smooth depth data
2. Spatially consistent depth data and poses
3. Robust 3D point tracking
4. Robust environment interaction

SpatialTrackerV2

It “kind of” works, lots of 3d artifacts. Need more time for full testing.





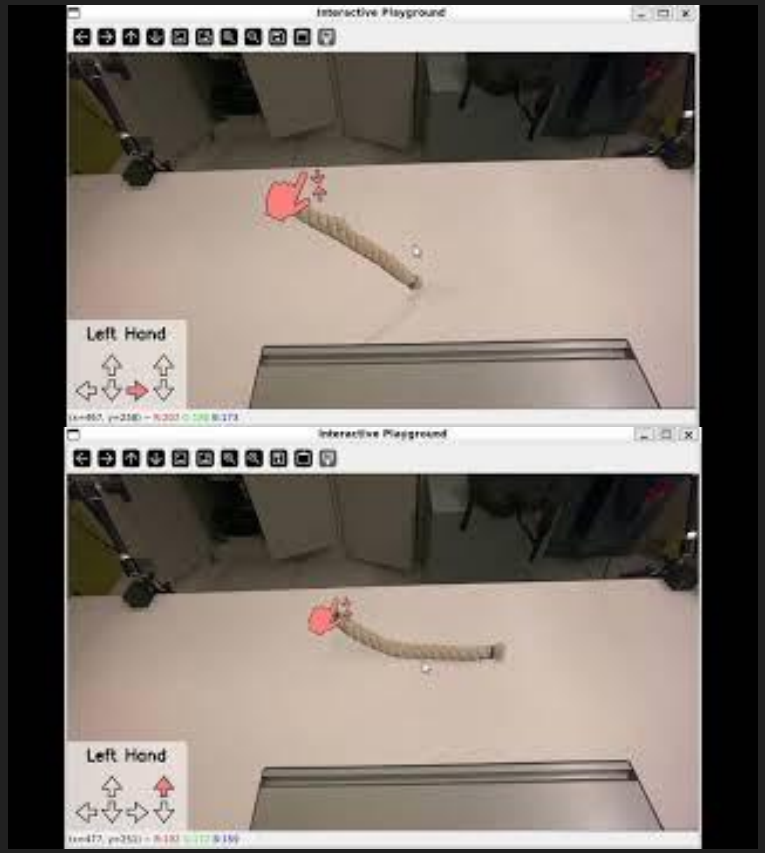
Proper Ground Collisions

Added new step:

1. Get ground plane from bounding box
2. Rotate extrinsics until ground is flat
3. Shift everything above ground

Resolved over-stiff model from pipeline

With proper ground interaction





Temporal Consistency

Key idea:

1. DA3 gives good geometry but flickering,
2. VDA gives poor geometry but good temporal smoothness

Train a model to learn the “temporal” component from VDA, but keep the “spatial” component

Model Architectures?

The model will need to be able to do the following things

- Learn meaningful DA3 structure
- Learn how to preserve spatial structure
- Learn temporal consistency from VDA
- Learn a way to modify DA3 depth predictions frame by frame
- Utilize information from DA3, VDA, and RGB inputs

Sounds like a image generative model?

Prior Research

VDA (2024)

- Extends DA2 to video by adding temporal attention layers at multiple decoder scales
- Introduces the TGM loss: frame to frame depth changes rather than absolute value
- Uses a segment-wise sliding window at inference with roll over on segments
- Dataset: real labeled depth data set from lidar

StableDPT (2025)

- Adds temporal attention into a DPT-style transformer backbone
- Encoder attends spatially per-frame, decoder attends temporally across frames at each spatial position
- Also adopts the TGM loss for temporal supervision
- Dataset: real labeled depth data set from lidar

Prior Research (closer to what we want)

NVDS / NVDS+ (Wang et al. 2023/2024)

- add-on stabiliser: any single-image depth model produces flickery disparities, NVDS refines them in a post-processing step
- cross-attention across a sliding window to learn temporal context + a optical flow model (GMFlow) for temporal loss calculation
- Coupled to DPT and MiDas as the depth generator
- Depends on metric depth
- Dataset: a **custom** dataset of 15k videos with of sensor ground truth

“Towards Practical Consistent Video Depth Estimation” (Li et al.): same problem

So what do we do…

Make our own model :)

Core Idea

Take DA3 input + RGB input -> encode into some latent space (force it to learn)

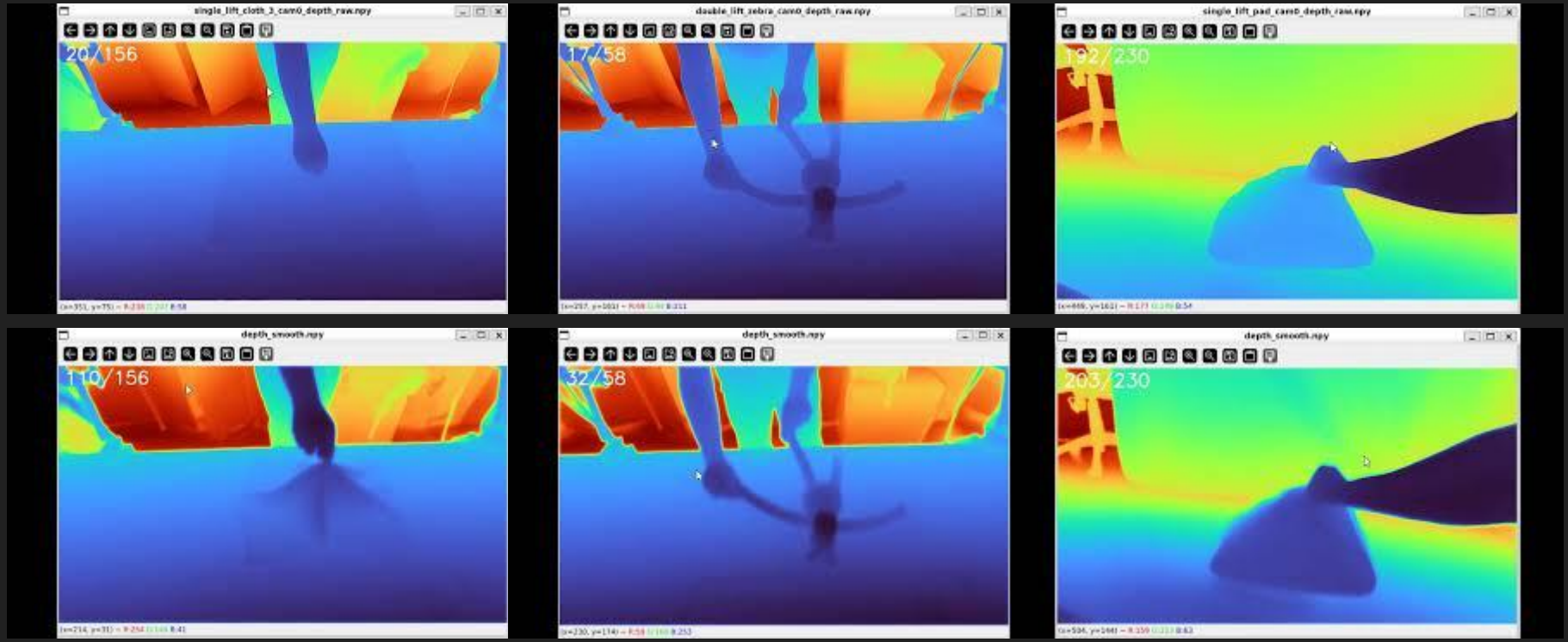
Decode latent space into smooth depth prediction

Loss will consist of two components

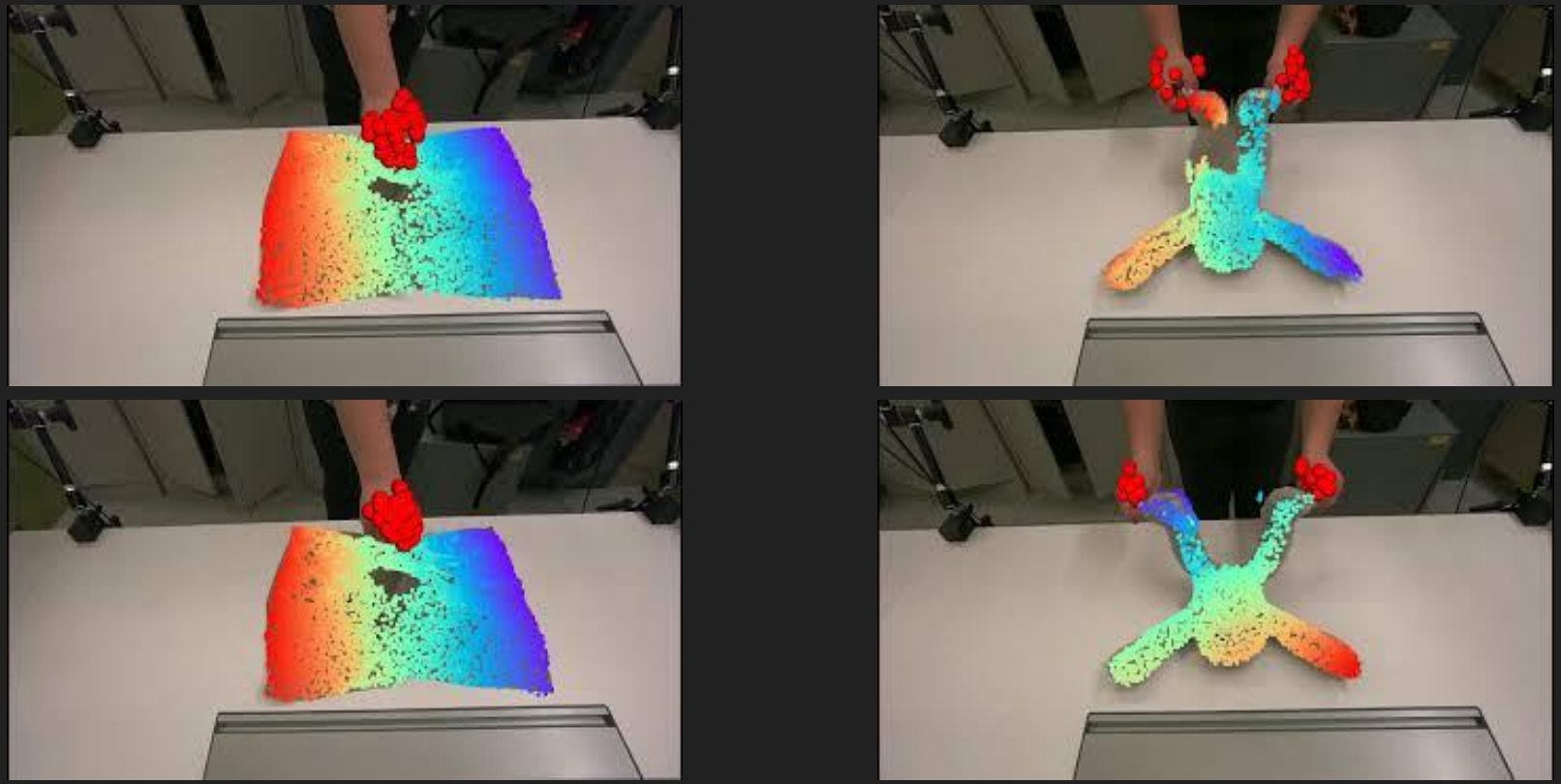
- Temporal penalties based on VDA baseline
- Geometric penalties based on DA3 baseline

Model will learn a **residual** to smooth depth instead of a whole depth map

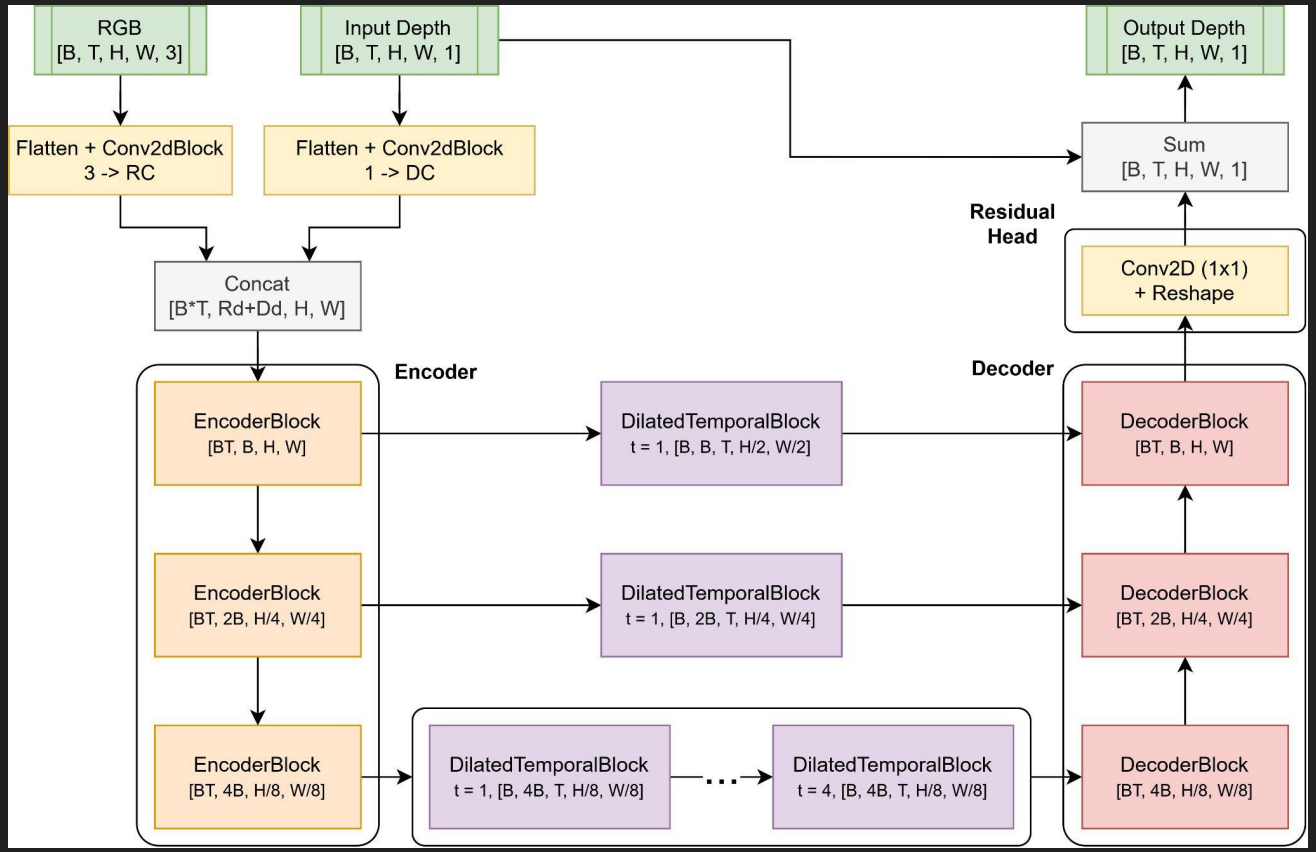
Results: raw depth



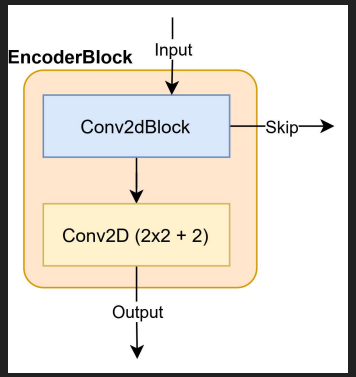
Results: after PhysTwin pipeline

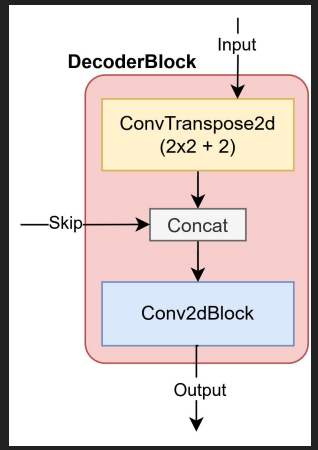


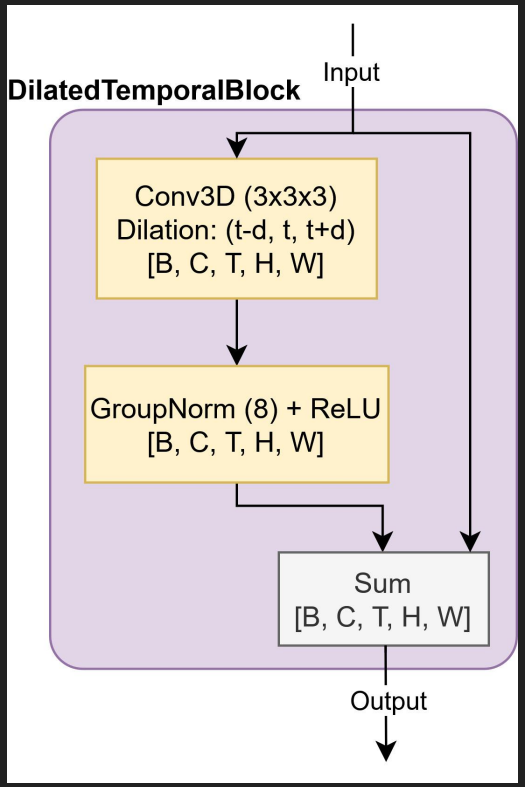
Temporally-Dilated Residual U-Net

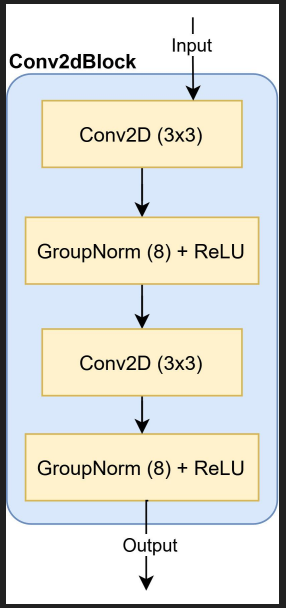


Some more details









Loss functions: TGM (temporal grad matching) loss

$$\mathcal{L}_{\text{tgm}}^{k} = \frac{1}{B_{\text{tgm}}^{k}} \cdot \mathbb{E}\left[\left|\Delta_k \hat{d} - \Delta_k d_{\text{vda}}\right|\right]$$

Forces the frame-to-frame depth changes in the output to match those of VDA.

- “Change in depth” of prediction should \approx that of VDA
- Used here only as a motion oracle, not a depth reference (is relative)
- k supervises short vs long term flicker
- Larger k can be too dominating, making everything flat

Loss functions: TV (temporal total variation) loss

$$\mathcal{L}_{\text{tv}}^{k} = \frac{1}{B_{\text{tv}}^{k}} \cdot \mathbb{E}\left[\left|\delta_k^2 \hat{d}\right| \cdot \exp\left(-10 \cdot \left|\delta_k^2 d_{\text{vda}}\right|\right)\right]$$

Penalises temporal acceleration (finite difference approx of second derivative)

- Input can have correct frame-to-frame changes (good TGM) but still oscillate unnaturally over longer spans
- Exp term “gates” out regions where VDA itself accelerates, so only unnatural oscillations are suppressed.
- Different k catches oscillations at different frequencies

Loss functions: fidelity loss

$$\mathcal{L}_{\text{fid}} = \frac{1}{B_{\text{fid}}} \cdot \mathbb{E}\left[\left|\hat{d}_{\text{aln}} - d_{\text{aln}}\right| \cdot g\right]$$

Keeps the output geometrically close to DA3.

Align scene to median scale

We use a VDA derived “gate” to enforce fidelity on stationary regions (high weight)

- If VDA says nothing is moving, any deviation in DA3 frames is likely flicker.
- In dynamic regions where VDA detects motion, the penalty is relaxed, the DA3 movement is probably real and we shouldn’t modify the geometry

Loss functions: geometric loss

$$\mathcal{L}_{\text{geom}} = \frac{1}{2} \cdot \mathbb{E}\left[\left(\left|\nabla_x \hat{d} - \nabla_x d_{\text{raw}}\right| + \left|\nabla_y \hat{d} - \nabla_y d_{\text{raw}}\right|\right) \cdot w_{\text{edge}}\right]$$

Ensures output edges match those of DA3

- gradients are computed directly via finite differences.
- Apply Sobel kernel on RGB frame to find “weight” mask. Gradient should match at edges, and doesn’t matter on flat areas.
- This is not normalized (i couldn’t figure it out)
- I am also not sure if this does much

Loss functions: all together

$$\mathcal{L}_{\text{tgm}} = \mathcal{L}_{\text{tgm}}^{k=1} + 0.2 \cdot \mathcal{L}_{\text{tgm}}^{k=2} + 0.05 \cdot \mathcal{L}_{\text{tgm}}^{k=3}$$

$$\mathcal{L}_{\text{tv}} = \mathcal{L}_{\text{tv}}^{k=1} + 0.2 \cdot \mathcal{L}_{\text{tv}}^{k=2}$$

$$\mathcal{L} = \lambda_{\text{fid}} \cdot \mathcal{L}_{\text{fid}} + \lambda_{\text{tgm}} \cdot \mathcal{L}_{\text{tgm}} + \lambda_{\text{geom}} \cdot \mathcal{L}_{\text{geom}} + \lambda_{\text{tv}} \cdot \mathcal{L}_{\text{tv}}$$

```
lambda_fidelity: float = 1.2
lambda_tgm: float = 0.5
lambda_tv: float = 0.5
lambda_geometric: float = 0.3
lambda_flicker: float = 0.0
```

These hyper-parameters are probably not optimal, needs way more testing!

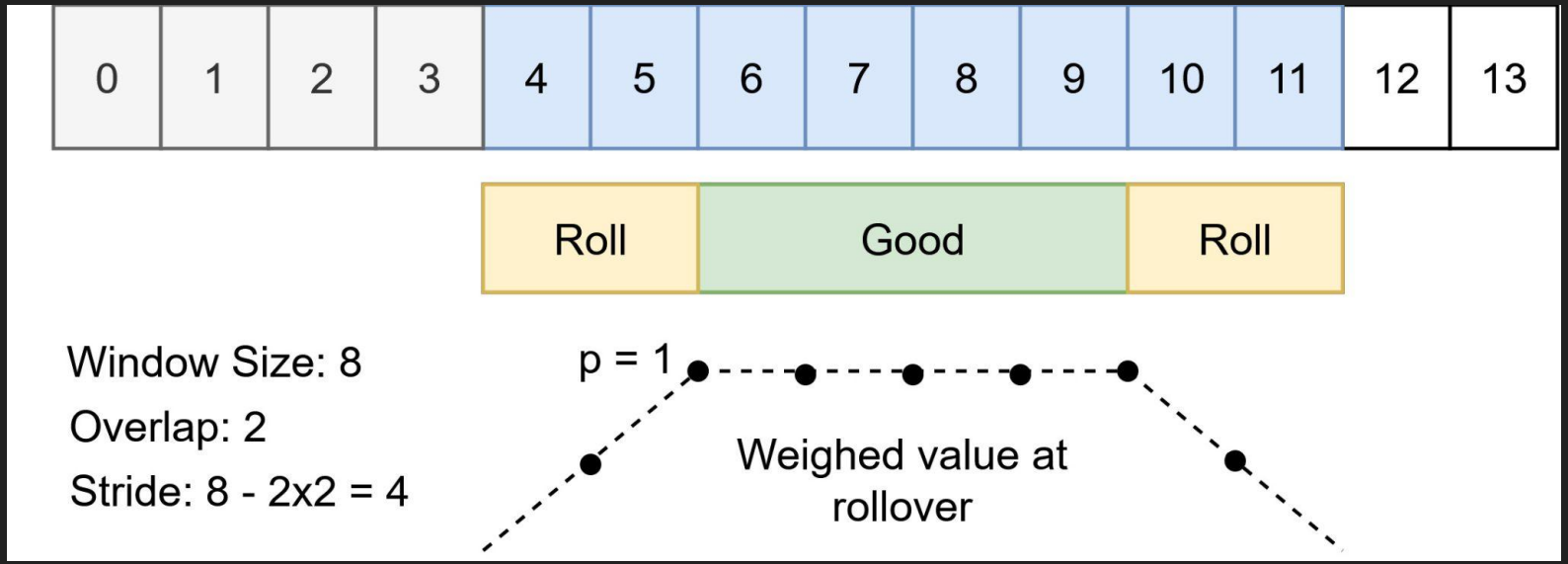
Training

~5M params, 50 epochs



Inference

Taken straight from VDA



So what does this mean?

1. We now have a model that we can drop on top of any flickery depth map
2. We don’t even need ground truth to train this depth smoother model
3. Works for any input / output resolution with “minimal” padding (no resize)
4. Fast
5. Need more testing, is it actually learning smoothness?
6. Smoothing makes depth map “blurry”
7. Training data is “worst case” with no fake sliding window

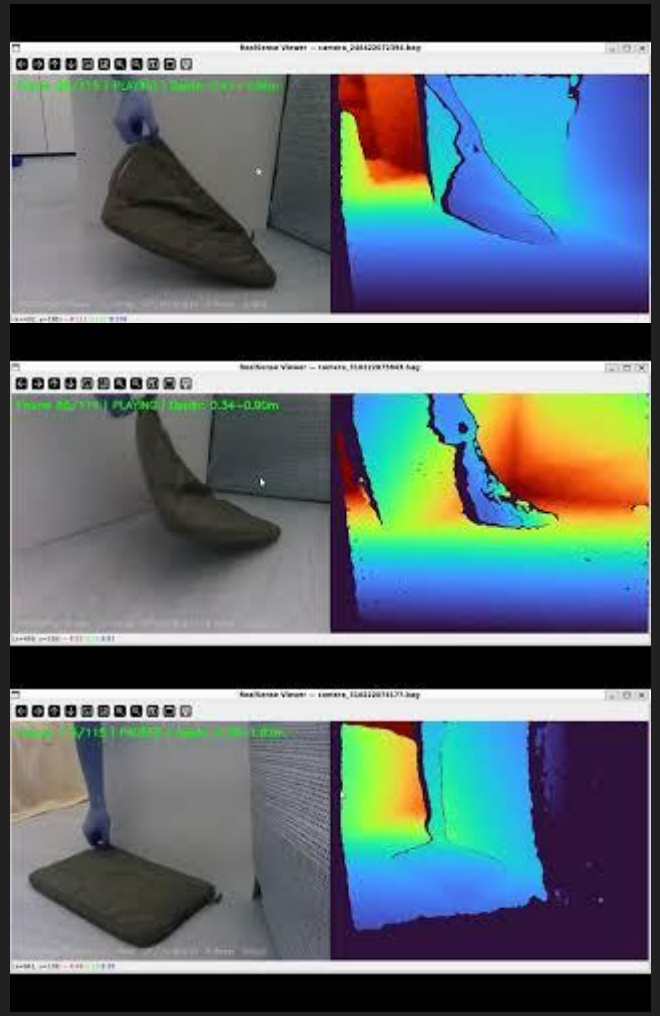
We might be ok with overfitting

- Overfit general model over each clip we want to smooth, “custom” smoothing model per clip. Only care about smoothing quality

New Data!

- 30 different cases
- Cloth, Pad, Bottles, Mugs, Stuffed Animals
- Currently running and testing
- Software aligned start times

Collected intrinsics and extrinsics manually



For next time

- Test smoothing model more (full pipeline runs)
- Test pipeline with newly recorded data

Overview of DigitalTwin V2

Background

PhysTwin, Physics-Informed Reconstruction and Simulation of Deformable Objects from Videos:

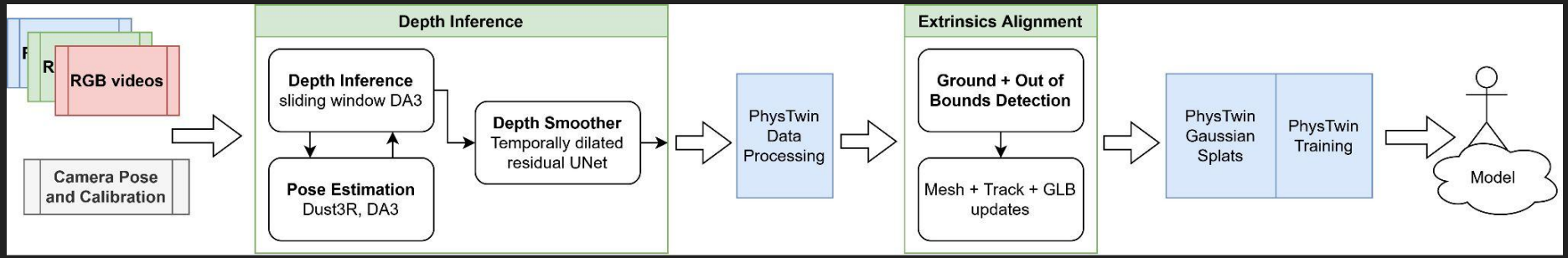
- Input: RGB + Depth videos
- Processes data to get tracked motion points of object and controller
- Trains warp model (spring mass system) given the points
- Result: interactive digital twin of object

DigitalTwin v2

The goals:

- Use pipeline with any RGB video input (without depth)
- Improve pipeline robustness / performance
- Improve physics model for more realistic simulation

New Pipeline structure



Green: new additions

Blue: OG phystwin components (with modification)

Depth Inference

“Fake” sliding window inference (Depth Anything 3)

- Produces non-metric geometrically consistent scene views given multiple camera poses (dynamic or positioned differently)
- “Fake”: duplicating multiple static views as “dynamic” (works for limited frames)
- Reduces major frame-to-frame inconsistencies

Dust3R or DA3 Pose Estimation

- Can infer the camera poses if not provided (worse results)
- Option to use Dust3R (better, pairwise) compared to DA3

Problem: Jittery depth videos!

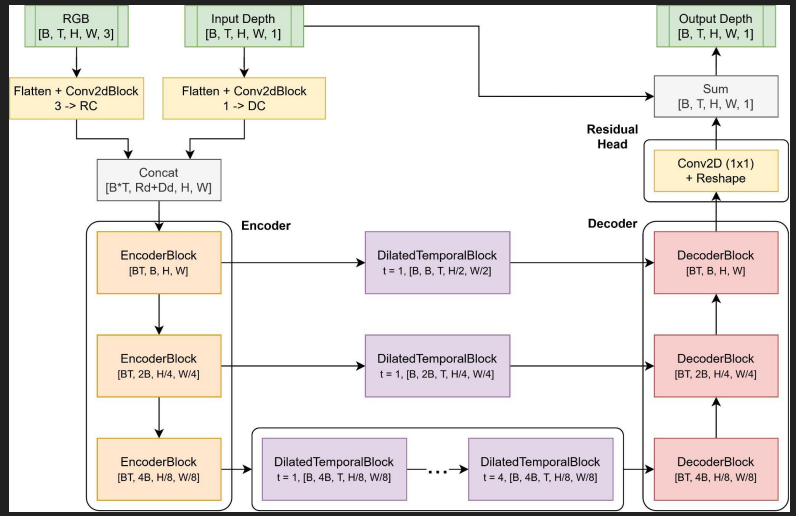
Temporal Depth Smoother

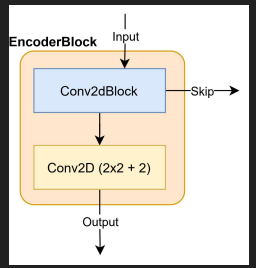
Goal: fix jittery depth map

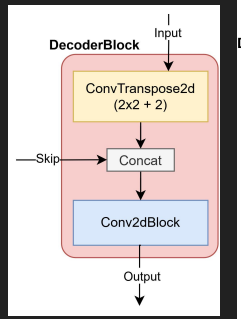
- Learn “smoothness” from VDA
- Learn “geometry” from DA3
- ML model that learns “corrections”

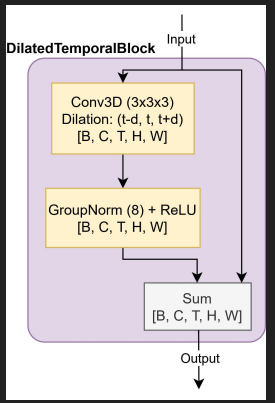
Model Specs

- Trained on PhysTwin + self collected Data
- ~4M params (limited by training compute)
- No streaming, but fast chunk-based inference









Extrinsics Alignment

Depth and Pose estimation may result in “improper” ground positions

- Ground plane needs to be $z = 0$ to work
- Effects model training (interaction with ground matters)

Solution:

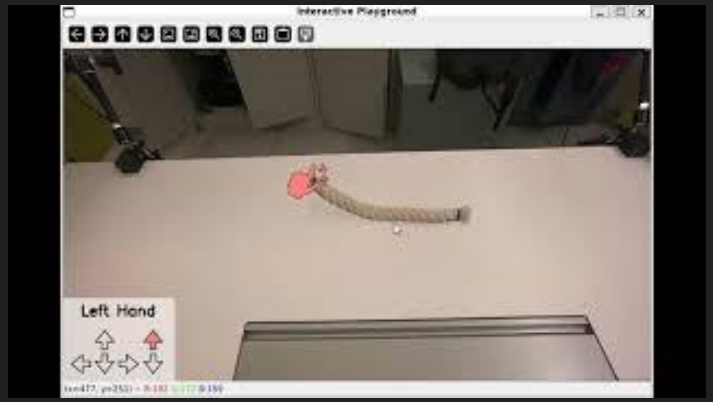
- Ground plane detection with RANSAC
- Align extrinsics such that plane is flat
- Propagate alignment changes to rest of pipeline

Misc Improvements

Other things

- Robust handling of different depth and RGB resolutions
- Robust shape prior alignment for non-water tight meshes
- Robust dense tracking with CoTracker for slightly separated masks
- Random QOL improvements

Results


Interactive Playground
Left Hand


Interactive Playground
Left Hand
Right Hand

Where are at now?

What we have

- A working pipeline to convert RGB videos into warp simulations

Here are some things to do

- More improvements required for depth smoother model
- More evaluation required on full pipeline
- More evaluation required with new data

Next Steps?

1. Clean up my code and publish on GitHub
2. …